

Yes. The key point is that **Function, Predicate, Consumer, Supplier, etc. are functional interfaces, but they do not themselves have “overloading” or “overriding” as special functional-interface features.** Java's normal method overloading and overriding rules still apply.

1. First understand the two concepts

Concept	Meaning	Decided at	Main purpose
Overloading	Same method name, different parameter list	Compile time	Multiple ways to call a method
Overriding	Subclass provides a new implementation of inherited method	Runtime	Polymorphic behavior

A simple example:

```
class Parent {  
  
    void process(int x) {  
        System.out.println("Parent int");  
    }  
}  
  
class Child extends Parent {  
  
    @Override  
    void process(int x) {  
        System.out.println("Child int");  
    }  
  
    void process(String x) {  
        System.out.println("Child String");  
    }  
}
```

Here:

```
process(int)    → overriding  
process(String) → overloading
```

2. Functional interface basics

A functional interface contains **exactly one abstract method.**

For example:

```
@FunctionalInterface
interface Function<T, R> {

    R apply(T t);
}
```

The important method is:

```
R apply(T t);
```

Similarly:

```
Predicate<T>
    boolean test(T t)
```

```
Consumer<T>
    void accept(T t)
```

```
Supplier<T>
    T get()
```

```
Function<T,R>
    R apply(T t)
```

These abstract methods are the methods implemented by lambda expressions.

3. Function<T,R>

Function<T,R> represents:

$T \rightarrow R$

It accepts one value and returns another value.

```
Function<String, Integer> length =
    str -> str.length();
```

```
System.out.println(length.apply("Java"));
```

Output:

4

Its abstract method is:

```
R apply(T t);
```

4. Overloading with Function

You can overload methods that accept different functional interfaces.

For example:

```
static void process(Function<String, Integer> f) {  
    System.out.println("Function");  
}  
  
static void process(Predicate<String> p) {  
    System.out.println("Predicate");  
}
```

Conceptually:

```
process(Function)  
process(Predicate)
```

are overloaded methods.

However, this creates an important Java lambda ambiguity.

This is problematic:

```
process(x -> x.length());
```

Why?

Because the compiler can potentially interpret the lambda against different functional-interface target types.

A lambda expression does not have a standalone type before target typing.

So **overloading methods with unrelated functional interfaces can create ambiguity**.

A cast can resolve it:

```
process((Function<String, Integer>) x -> x.length());
```

or:

```
process((Predicate<String>) x -> x.length() > 3);
```

5. Function method overloading

You can also overload ordinary methods inside a class.

```
class Calculator {  
  
    int calculate(int x) {  
        return x * 2;  
    }  
  
    double calculate(double x) {  
        return x * 2;  
    }  
  
    int calculate(int x, int y) {  
        return x + y;  
    }  
}
```

This has nothing specifically to do with Function.

The same Java overloading rules apply.

```
calculate(10);  
calculate(10.5);  
calculate(10, 20);
```

The compiler selects the appropriate method.

6. Function overriding

You can create your own functional interface extending another functional interface.

For example:

```
@FunctionalInterface  
interface MyFunction<T, R> extends Function<T, R> {  
  
    @Override  
    R apply(T t);  
}
```

Here:

```
R apply(T t);
```

is overriding/redeclaring the inherited abstract method.

You can use it:

```
MyFunction<String, Integer> f =  
    str -> str.length();  
  
System.out.println(f.apply("Java"));
```

Output:

4

The lambda implements apply().

7. Important distinction: lambda does NOT override a method in the normal inheritance sense

Consider:

```
Function<String, Integer> f =  
    str -> str.length();
```

It is tempting to say:

"The lambda overrides Function.apply()."

For conceptual learning, you can say the lambda **provides the implementation of the functional interface's abstract method**.

But technically, a lambda is not a subclass declaration such as:

```
class MyFunction implements Function<String, Integer> {  
  
    @Override  
    public Integer apply(String str) {  
        return str.length();  
    }  
}
```

The explicit class uses ordinary interface implementation and overriding.

The lambda uses the functional-interface mechanism.

8. Predicate

Predicate<T> represents:

$T \rightarrow \text{boolean}$

Its abstract method is:

```
boolean test(T t);
```

Example:

```
Predicate<Integer> isEven =  
    n -> n % 2 == 0;
```

```
System.out.println(isEven.test(10));
```

Output:

```
true
```

9. Overloading with Predicate

You can overload ordinary methods using Predicate:

```
static void check(Predicate<String> predicate) {  
    System.out.println("String predicate");  
}  
  
static void check(Predicate<Integer> predicate) {  
    System.out.println("Integer predicate");  
}
```

But **this does not compile**.

Why?

Because after type erasure:

```
Predicate<String>  
Predicate<Integer>
```

both become:

```
Predicate
```

Therefore the methods have the same erased signature.

This is a very important rule:

You cannot overload methods solely by changing generic type arguments.

For example:

```
void process(List<String> list)
void process(List<Integer> list)
```

is also illegal because of type erasure.

10. Predicate and overloaded methods with different functional interfaces

This is possible:

```
static void process(Predicate<String> p) {
    System.out.println("Predicate");
}

static void process(Function<String, Boolean> f) {
    System.out.println("Function");
}
```

These have different raw types:

Predicate
Function

so the declarations themselves are legal.

But:

```
process(x -> x.length() > 3);
```

may be ambiguous because the lambda can potentially target either interface.

Explicit casting solves it:

```
process((Predicate<String>) x -> x.length() > 3);
```

or:

```
process((Function<String, Boolean>) x -> x.length() > 3);
```

11. Predicate overriding

You can extend Predicate:

@FunctionalInterface

```
interface MyPredicate<T> extends Predicate<T> {  
  
    @Override  
    boolean test(T t);  
}
```

Then:

```
MyPredicate<Integer> p =  
    n -> n > 100;  
  
System.out.println(p.test(200));
```

Output:

true

Here MyPredicate inherits the contract of Predicate.test().

12. Predicate's default methods

Predicate is especially interesting because it has default methods:

```
and()  
or()  
negate()
```

For example:

```
Predicate<Integer> greaterThan10 =  
    n -> n > 10;  
  
Predicate<Integer> lessThan100 =  
    n -> n < 100;  
  
Predicate<Integer> between =  
    greaterThan10.and(lessThan100);
```

Now:

```
between.test(50);
```

returns:

true

The relationship is:

Predicate

```
|  
+--- abstract test()  
|  
+--- default and()  
+--- default or()  
+--- default negate()
```

These default methods are **not additional abstract methods**, so Predicate remains a functional interface.

13. Consumer

Consumer<T> represents:

$T \rightarrow \text{void}$

It accepts an object but does not return a result.

Its abstract method is:

```
void accept(T t);
```

Example:

```
Consumer<String> printer =  
    s -> System.out.println(s);
```

```
printer.accept("Java");
```

Output:

```
Java
```

14. Consumer overloading

You can overload methods like this:

```
static void execute(Consumer<String> c) {  
    System.out.println("String consumer");  
}
```

```
static void execute(Function<String, Integer> f) {
```

```
    System.out.println("Function");  
}
```

The declarations are legal.

But this:

```
execute(x -> System.out.println(x));
```

can be affected by lambda target typing and overload resolution.

The safer approach when ambiguity occurs is:

```
execute((Consumer<String>) x -> System.out.println(x));
```

15. Consumer overriding

You can extend Consumer:

```
@FunctionalInterface  
interface MyConsumer<T> extends Consumer<T> {  
  
    @Override  
    void accept(T t);  
}
```

Then:

```
MyConsumer<String> consumer =  
    s -> System.out.println(s.toUpperCase());  
  
consumer.accept("java");
```

Output:

JAVA

16. Consumer's default method

Consumer has:

```
andThen()
```

Example:

```
Consumer<String> print =
```

```
s -> System.out.println("Print: " + s);
```

```
Consumer<String> log =  
    s -> System.out.println("Log: " + s);
```

```
Consumer<String> combined =  
    print.andThen(log);
```

```
combined.accept("Java");
```

Output:

Print: Java

Log: Java

Execution:

```
combined.accept("Java")
```

↓

```
print.accept("Java")
```

↓

```
log.accept("Java")
```

17. Supplier

Supplier<T> represents:

$() \rightarrow T$

It takes no argument and produces a value.

Its abstract method is:

```
T get();
```

Example:

```
Supplier<String> supplier =  
    () -> "Java";
```

```
System.out.println(supplier.get());
```

Output:

Java

18. Supplier overloading

Consider:

```
static void execute(Supplier<String> s) {  
    System.out.println("Supplier");  
}  
  
static void execute(Callable<String> c) {  
    System.out.println("Callable");  
}
```

Both functional interfaces have:

$() \rightarrow \text{String}$

So:

```
execute(() -> "Java");
```

is ambiguous.

The lambda itself doesn't inherently say:

I am Supplier

or:

I am Callable

You can explicitly cast:

```
execute((Supplier<String>) () -> "Java");
```

or:

```
execute((Callable<String>) () -> "Java");
```

This is one of the most important practical issues with functional-interface overloading.

19. Supplier overriding

You can extend Supplier:

```
@FunctionalInterface  
interface MySupplier<T> extends Supplier<T> {
```

```

    @Override
    T get();
}

```

Usage:

```

MySupplier<String> supplier =
    () -> "Hello Java";

System.out.println(supplier.get());

```

20. Function vs Predicate vs Consumer vs Supplier

The easiest way to remember them:

Interface	Input	Output	Abstract method
Function<T,R>	T	R	apply(T)
Predicate<T>	T	boolean	test(T)
Consumer<T>	T	none	accept(T)
Supplier<T>	none	T	get()

Think of them as:

Function
 $T \longrightarrow R$

Predicate
 $T \longrightarrow \text{boolean}$

Consumer
 $T \longrightarrow \text{void}$

Supplier
 $\longrightarrow T$

21. Where overloading becomes especially interesting

Suppose we have:

```

class Processor {

    void process(Function<String, Integer> f) {

```

```

    System.out.println("Function");
}

void process(Predicate<String> p) {
    System.out.println("Predicate");
}

void process(Consumer<String> c) {
    System.out.println("Consumer");
}

void process(Supplier<String> s) {
    System.out.println("Supplier");
}
}

```

Then:

```
Processor p = new Processor();
```

Consider:

```
p.process(s -> s.length());
```

The compiler can identify this as a `Function<String,Integer>` because the lambda returns an integer.

But:

```
p.process(s -> System.out.println(s));
```

strongly corresponds to `Consumer` because it produces no value.

And:

```
p.process(() -> "Java");
```

corresponds to `Supplier` because it takes no parameter.

However, where multiple functional interfaces have compatible shapes, overload resolution can become ambiguous.

22. A very important example: Predicate vs Function

Consider:

```
void process(Predicate<String> p) {
```

```

    System.out.println("Predicate");
}

void process(Function<String, Boolean> f) {
    System.out.println("Function");
}

```

Now:

```
process(x -> x.length() > 5);
```

Both interfaces can conceptually accept:

String → boolean/Boolean

Therefore the lambda may not provide enough information to distinguish the intended target.

Use:

```
process((Predicate<String>) x -> x.length() > 5);
```

or:

```
process((Function<String, Boolean>) x -> x.length() > 5);
```

23. Overriding through a functional interface hierarchy

Consider:

```

interface Parent<T> {
    T process(T value);
}

```

```

@FunctionalInterface
interface Child<T> extends Parent<T> {

    @Override
    T process(T value);
}

```

Now:

```

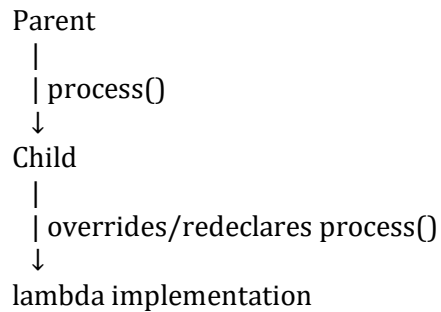
Child<String> child =
    s -> s.toUpperCase();

```

The lambda implements:

process()

The inheritance relationship is:



24. Can a functional interface have overloaded abstract methods?

No.

This is one of the most important rules.

This is not a functional interface:

```
interface MyFunction {  
    void process(int x);  
    void process(String x);  
}
```

There are two abstract methods:

```
process(int)  
process(String)
```

Therefore:

✗Not a functional interface

You cannot do:

```
MyFunction f = x -> System.out.println(x);
```

because Java does not know which abstract method the lambda is implementing.

25. Can a functional interface have overloaded default methods?

Yes.

A functional interface can have many default and static methods, provided it has only one abstract method.

For example:

```
@FunctionalInterface
interface MyFunction {

    int apply(int x);

    default void print(int x) {
        System.out.println(x);
    }

    default void print(String x) {
        System.out.println(x);
    }

    static void info() {
        System.out.println("MyFunction");
    }
}
```

This is valid.

Why?

Because:

Abstract methods = 1
Default methods = 2
Static methods = 1

Therefore it remains a functional interface.

26. Built-in functional interfaces follow this model

For example, Predicate conceptually looks like:

```
@FunctionalInterface
public interface Predicate<T> {

    boolean test(T t);

    default Predicate<T> and(Predicate<? super T> other) {
```

```

    ...
}

default Predicate<T> negate() {
    ...
}

default Predicate<T> or(Predicate<? super T> other) {
    ...
}

static <T> Predicate<T> isEqual(Object targetRef) {
    ...
}
}

```

Notice:

One abstract method:

test()

Multiple default methods:

and()

negate()

or()

Static method:

isEqual()

Therefore Predicate is still functional.

27. Overloading vs overriding in these interfaces

This distinction is extremely important.

Overloading

void process(Function<String, Integer> f)

void process(Predicate<String> p)

Same method name:

process()

Different parameter types:

Function
Predicate

Therefore:

OVERLOADING

Overriding

```
interface MyFunction extends Function<String, Integer> {  
  
    @Override  
    Integer apply(String value);  
}
```

Here:

```
Function.apply()  
  ↓  
MyFunction.apply()
```

Therefore:

OVERRIDING / REDECLARATION

28. Real-world example: Employee processing

Suppose we have:

```
class Employee {  
  
    String name;  
    double salary;  
    boolean active;  
  
    Employee(String name, double salary, boolean active) {  
        this.name = name;  
        this.salary = salary;  
        this.active = active;  
    }  
}
```

Now use all four functional interfaces.

Predicate — filtering

Predicate<Employee> highSalary =

```
e -> e.salary > 100000;
```

Meaning:

Employee → boolean

Function — transformation

```
Function<Employee, Double> getSalary =  
    e -> e.salary;
```

Meaning:

Employee → Double

Consumer — action

```
Consumer<Employee> printEmployee =  
    e -> System.out.println(e.name);
```

Meaning:

Employee → void

Supplier — object/value creation

```
Supplier<Employee> createEmployee =  
    () -> new Employee("Anurag", 50000, true);
```

Meaning:

() → Employee

So the business pipeline becomes:

Supplier
↓
creates Employee

Predicate
↓
checks Employee

Function
↓

extracts/transforms Employee

Consumer



uses/displays Employee

29. Combining them

```
Supplier<Employee> supplier =  
    () -> new Employee("Anurag", 120000, true);
```

```
Predicate<Employee> highSalary =  
    e -> e.salary > 100000;
```

```
Function<Employee, Double> getSalary =  
    e -> e.salary;
```

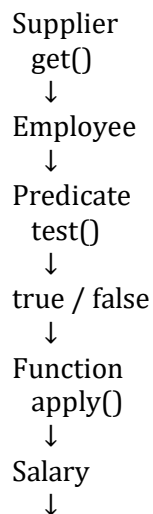
```
Consumer<Double> printSalary =  
    salary -> System.out.println("Salary = " + salary);
```

Execution:

```
Employee emp = supplier.get();
```

```
if (highSalary.test(emp)) {  
    Double salary = getSalary.apply(emp);  
    printSalary.accept(salary);  
}
```

Flow:



Consumer
accept()
↓
Output

This is a very common functional-programming pattern.

30. The biggest interview point

Do **not** say:

"Function, Predicate, Consumer and Supplier support method overloading and overriding."

A more accurate answer is:

Function, Predicate, Consumer and Supplier are functional interfaces. Their methods participate in Java's normal method overloading and overriding rules. The functional-interface requirement is that the interface have exactly one abstract method; it may additionally contain multiple default and static methods.

And remember:

Functional Interface

- |
 - +— exactly ONE abstract method
 - |
 - +— zero or more default methods
 - |
 - +— zero or more static methods
 - |
 - +— can participate in inheritance
 - |
 - +— can be implemented by lambda/method reference
-

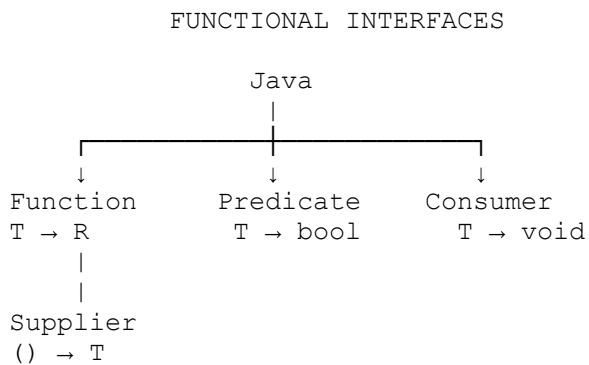
31. Complete comparison

Concept	Function	Predicate	Consumer	Supplier
Input	1	1	1	0
Output	1	boolean	void	1
Abstract method	apply()	test()	accept()	get()
Can have default methods?	Yes	Yes	Yes	Yes

Can have static methods?	Yes	Yes	Yes	Yes
Can be extended?	Yes	Yes	Yes	Yes
Can be overridden in child interface?	Yes	Yes	Yes	Yes
Can be used with lambda?	Yes	Yes	Yes	Yes
Can be used with method reference?	Yes	Yes	Yes	Yes
Can have multiple abstract methods?	No	No	No	No
Can ordinary methods overload these interfaces?	Yes	Yes	Yes	Yes
Can overload only by generic parameter?	No	No	No	No

Final mental model

Think of the four interfaces as four different **function shapes**:



And distinguish the two Java mechanisms:

OVERLOADING

Same method name
Different parameter list
Compile-time decision

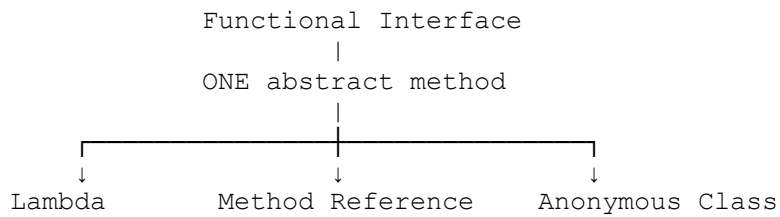
process(Function)
process(Predicate)
process(Consumer)
process(Supplier)

OVERRIDING

Inherited method
Child provides/redeclares implementation
Runtime polymorphism

Function.apply()
↓
ChildFunction.apply()

The most important functional-interface rule is:



while **default/static methods do not count toward the single-abstract-method requirement.**